

TRACK EXPENSES!

dreamstime.



Campus Coin

Smart Spending Student Style

Software Requirements Specification
Version 1.0

Project Name: Campus Coin
Theme: NextGen BudgetBee
Category: End-to-End Web Solutions

Table of Contents

1.1	Background and Necessity for the Web Application	3
1.2	Proposed Solution	3
1.3	Purpose of the Document	5
1.4	Scope of Project	5
1.5	Constraints	5
1.6	Functional Requirements	6
1.7	Non-Functional Requirements.....	11
1.8	Interface Requirements	12
1.9	Project Deliverables	15

1.1 Background and Necessity for the Web Application

College and university students routinely receive money from multiple, irregular sources such as monthly allowance from family, part-time or gig-work income, scholarships, and occasional gifts, yet rarely track where that money actually goes. Generic personal-finance apps are built for salaried adults with fixed pay cycles and bank integrations, and they are often too complex, subscription-gated, or simply irrelevant to a student's spending patterns (canteen food, hostel expenses, textbooks, transport, subscriptions, and social outings).

There is a clear demand for a lightweight, student-first Web application that makes it effortless to log income and expenses, view spending by category, and receive plain-language guidance on how to save more without requiring a linked bank account or financial expertise. An application is required to fulfil this demand by offering a dynamic, responsive platform which provides following features:

- Fast entry of income and expenses with student-relevant categories
- Personalized, easy-to-follow saving tips generated from the student's own transaction history
- Optional AI assistance that automatically categorizes expenses and summarizes monthly spending in plain language

Built with modern front-end technologies backed by a robust database layer, the Web application should ensure a smooth, reliable experience across devices, making it ideal for individual students, hostel communities, and campus financial-literacy initiatives.

1.2 Proposed Solution

To close the gap between generic finance apps and the realities of student life, this document proposes the development of a fully functional Web application named Campus Coin, a responsive, interactive student



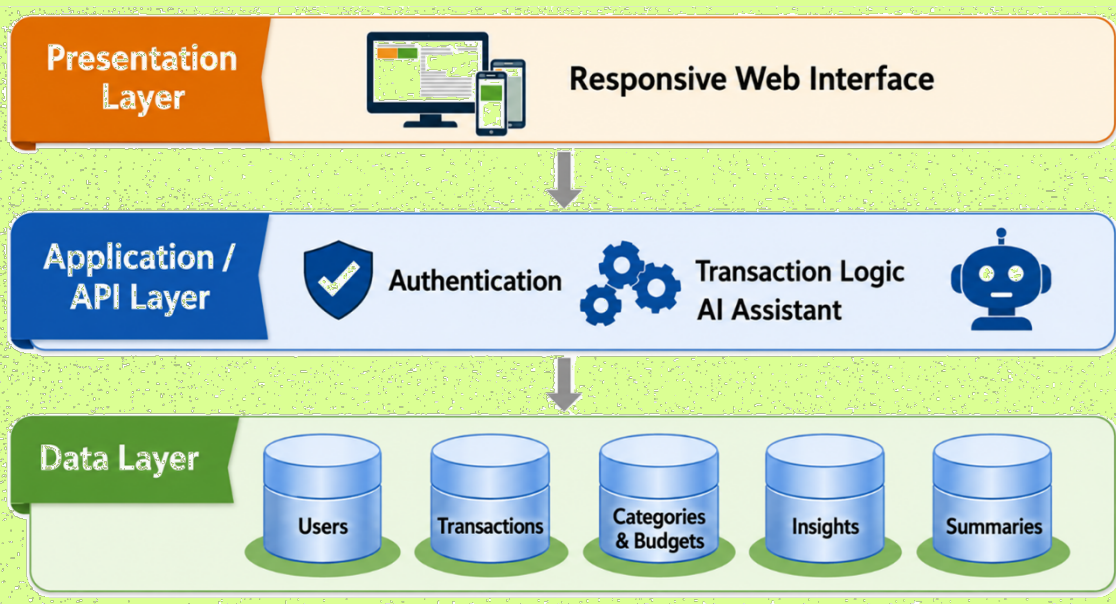
budget and expense tracker designed for college and university students.

The application offers a seamless experience where a student can register, log in, and immediately record income such as allowance, part-time job earnings, scholarships, and gifts, and expenses such as food, transport, hostel or rent, academics, subscriptions, entertainment, and miscellaneous. Every transaction is stored in a central database, which powers category-wise breakdowns, month-over-month trend charts, and a personalized tips engine that reacts to the student's own habits. An optional AI-driven assistant can help categorize expenses automatically as they are entered. It can generate a friendly, actionable monthly insight summary, for example flagging that food delivery spending rose sharply, and suggesting a simple and achievable adjustment.

Architecture Overview

The high-level architecture follows a standard three-tier design:

- **Presentation Layer:** Responsive Web front end (dashboard, entry forms, charts, insights feed)
- **Application/API Layer:** Handles authentication, transaction logic, report generation, and optional AI-assistant requests
- **Data Layer:** Relational database storing users, transactions, categories, budgets, and generated insights

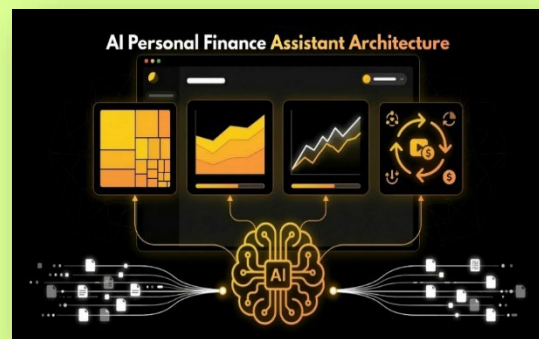


1.3 Purpose of the Document

This document presents a detailed description of the **Campus Coin** application, explaining its features, purpose, scope, and limitations. It is intended for both stakeholders and developers of the application and serves as the basis for design, development, and evaluation of the project.

1.4 Scope of Project

Campus Coin is a full-stack Web application designed to help students take control of their personal finances by logging income and expenses, categorizing spending, and reviewing their financial habits through monthly reports. It offers a personalized experience through user registration and login, quick transaction entry, a category management system, an interactive dashboard with charts, and a saving-tips engine driven by the student's own transaction history.



The application includes a backend system to support data storage, session management, budget-goal tracking, and report generation. Users can add and edit transactions, set monthly budgets per category, view spending trends over time, receive optional AI-generated categorization suggestions, and read AI-generated monthly spending insights with simple, actionable advice. Built with modern technologies, the platform is scalable, responsive, and adaptable for individual students, student communities, and campus financial-literacy programs. Administrative functionality will also be part of the application, giving oversight of categories, users, and system-wide usage statistics.

1.5 Constraints

Development of the **Campus Coin** Web application must adhere to several constraints to ensure successful implementation and operation. Technically, the application must be compatible with major Web browsers and responsive across

desktop, tablet, and mobile devices. Constraints may also arise around data storage, data synchronization, and backup procedures for transaction history.

Since **Campus Coin** does not integrate with real banking systems, all income and expense data is manually entered or imported via simple file upload, for example CSV. The application will not perform actual bank-account verification, real payment processing, or handle real monetary transactions. These aspects are beyond the scope of the application. Any AI-generated categorization or insight is advisory in nature and must be presented as a suggestion the student can review or override, not as certified financial advice.

1.6 Functional Requirements



The **Campus Coin** Web application will offer a complete and student-centric budgeting experience with dynamic front-end features and robust backend support. It will not only cover essential functionality, but also implement advanced capabilities to enhance personalization, insight generation, and interactivity.

User Authentication and Management

- Student registration and login; separate, direct-access administrator login
- Secure session management
- Password recovery and reset through email verification or a tokenized link
- User profile creation with editable fields: name, academic year, monthly allowance baseline, and savings goal
- Optional bulk import of historical transactions from CSV files

Category Management

Registered users should be able to create their personal categories under category type income and expenses

Income Categories	Expense Categories
• Allowance • Part-time Job • Scholarship • Gift • Other Income	• Food • Transport • Hostel/Rent • Academics (books, tuition, stationery) • Subscriptions (streaming, apps) • Entertainment (movies, outings) • Miscellaneous

Add, edit, and delete personal categories under 'Manage Own Categories'.

Personalized Dashboard

- Displays personalized greeting, current month balance (income vs. expense), and quick-add buttons
- Recommends saving tips and highlights based on the student's own transaction history
- Displays dynamic widgets such as 'This Month's Top Category' and 'Budget vs. Actual'

Income and Expense Logging

- Quick-add form for income (allowance, part-time job, scholarship, gift, and other income) and expenses (food, transport, hostel/rent, academics, subscriptions, entertainment, and other expenses)
- Supports recurring entries (for example, monthly allowances and subscription charges)
- Edits and deletes transactions while retaining the full history

Optional AI-Driven Expense Categorization Assistant

- Uses AI/ML techniques to suggest a category automatically as the student types a transaction description (for example, 'Campus Cafe': Food).
- Learns from the student's corrections over time to improve future suggestions
- Allows manual override of any AI-suggested category

- Provides batch-categorization suggestions when importing CSV transaction history

Monthly Reports

- Provides a category-wise report of monthly spending
- Provides an income-versus-expense report covering the last six months
- Provides daily and weekly spending summaries for the current month
- Filters reports by date range, category, or income source
- Exports the monthly report as a PDF or image for personal records

Optional AI-Generated Monthly Spending Insights

- Uses AI tools to analyze the month's transactions and generate a short, plain-language narrative summary highlighting notable patterns.
- Flags categories with above-average growth compared to the student's own trend (for example, 'Food delivery spending rose 40% this month')
- Provides simple and actionable advice tied to the flagged pattern (for example, a suggested weekly cap or a lower-cost alternative)
- Stores insight history so students can review past months' summaries

Personalized Saving Tips Engine

- Generates tips from the database by comparing current spending against the student's historical averages and set budget goals
- Ranks tips by potential savings impact and displays the top few on the dashboard
- Allows students to dismiss or 'pin' tips they find useful

Budget Goals and Alerts

- Sets a monthly budget per category (for example, food: 30 USD)
- Shows budget consumption in real time through progress bars
- Sends an in-app notification when a category nears or exceeds its budget

Bookmarking, Notes, and Sharing

- Bookmarks a saving tip or monthly insight for later reference
- Optionally exports monthly reports or savings summaries as PDFs or shares them by email

Admin Control Panel

Add/edit/remove:

- Default expense/income categories available to all students
- System-wide announcement or tip templates
- User accounts (view, disable, or reset)
- View usage statistics for active users, total transactions logged, and most-used categories

Optional System Intelligence (Advanced UX)

- Tracks recently viewed and recently edited transactions across sessions
- Forecasts for the upcoming month based on historical trends (optional)
- Detects and flags are unusually large or duplicate transactions

Accessibility and UI Enhancements

- Provides a dark-mode toggle and font-size adjustment for accessibility
- Provides breadcrumbs for clear navigation across dashboard sections
- Provides smooth transitions and loading indicators while charts and insights are generated

Important Note Regarding AI Usage:

You are encouraged to use AI-powered tools (such as AI-assisted Website builders, UI/UX design tools, code assistants, and image-generation tools) to enhance productivity and creativity. However, AI should be used as a supporting aid rather than a substitute for your own design, development, and problem-solving skills.

Do NOT rely on completely ready-made Website templates for your project, as this will adversely affect your evaluation. Your Web application's design, structure, and implementation should primarily reflect your own skills and understanding. Do NOT submit AI-generated code or content without meaningful modification and understanding. AI-generated suggestions may be used for guidance, learning, debugging, or improving productivity, but the final solution should demonstrate your own effort, logic, and implementation.

Acknowledge all the AI tool(s) used (for example, Copilot, Canva AI, Figma AI, Uizard, or similar) in your project documentation or submission.

During evaluation, judges may ask participants to explain their design decisions, implementation approach, and code. You are, therefore, expected to understand and be able to justify all aspects of your submitted work.

Do not use AI tools to fully produce ready-made documentation. This is strictly forbidden.

Bottomline: AI is your **assistant**, not your **developer**. Your knowledge, creativity, and coding skills should drive the project.

1.7 Non-Functional Requirements

There are several non-functional requirements that should be fulfilled by the application. They include:

- **Safe to use:** The application should not result in any malicious downloads or unnecessary file downloads.
- **Accessibility:** The application should have clear and legible fonts, user-interface elements, and navigation elements.
- **User-friendliness:** The application should be easy to navigate through clear menus and easy to understand, especially for first-time users logging their first transaction.
- **Operability:** The application should be reliable and efficient.
- **Performance:** The application should demonstrate high performance through speed and throughput, with minimal load time when rendering charts and AI-generated insights.
- **Scalability:** The application architecture and infrastructure should be designed to handle increasing user traffic, growing transaction volume, and feature expansions.
- **Security:** The application should implement adequate security measures such as authentication. Only registered users can access their own transaction history.
- **Availability:** The application should be available 24/7 with minimum downtime.
- **Compatibility:** The application should be compatible with the latest browsers and various devices.



These are the bare minimum expectations from the project. It is a must to implement the FUNCTIONAL and NON-FUNCTIONAL requirements given in this SRS. Once they are complete, you can use your own creativity and imagination to add more features if required.

1.8 Interface Requirements

Hardware

Intel Core i5/i7 Processor or higher
16 GB RAM or higher
Color SVGA
500 GB Hard Disk space or higher
Mouse
Keyboard

Software

IDE: Appropriate IDE as per the platform

Frontend: HTML5, CSS3, Bootstrap, ReactJS/AngularJS/Angular, JavaScript, jQuery, and XML

Backend: Java SDK with Apache NetBeans or Eclipse, Jakarta EE
OR
C# with ASP.NET MVC and ASP.NET MVC Core (optional), Visual Studio IDE
OR
PHP with Laravel Framework
OR
Python with Flask or Django
OR
MongoDB, Express.js, Angular, Node.js
OR
MongoDB, Express.js, React, Node.js

Database: MySQL/SQL Server/MongoDB/JSON

AI/ML Support: Services or APIs for expense categorization (like OCR) and insight generation [optional]

AI ChatBot: AI assistant/chatbot may be implemented using tools such as tawk.to or Tidio

Database Design

Based on the given specifications, suitable entities, attributes, and relationships should be defined. Some entities along with their indicative attributes are shown here as examples you do not have to adhere to these structures and can design your own table structure with different columns.

User

Attribute	Type	Description
user_id (PK)	INT/UUID	Unique identifier for the student
name	VARCHAR	Full name of the student
email	VARCHAR	Login email, unique
password_hash	VARCHAR	Encrypted password
academic_year	VARCHAR	Optional profile field
monthly_savings_goal	DECIMAL	Target savings amount set by the user
created_at	DATETIME	Account creation timestamp

Category

Attribute	Type	Description
category_id (PK)	INT	Unique identifier for the category
name	VARCHAR	Category name For example, Food, Transport, Hostel, Academics
type	ENUM	Category type such as 'income' or 'expense'
is_default	BOOLEAN	Whether it is a system default or user-created

Transaction

Attribute	Type	Description
transaction_id (PK)	INT/UUID	Unique identifier for the transaction
user_id (FK)	INT/UUID	References User
category_id (FK)	INT	References Category
amount	DECIMAL	Transaction amount
type	ENUM	Transaction type, such as 'income' or 'expense'
description	VARCHAR	Free-text note entered by the user
ai_suggested_category	INT	Category suggested by the AI assistant
date	DATE	Date of the transaction
created_at	DATETIME	Record creation timestamp

Budget

Attribute	Type	Description
budget_id (PK)	INT	Unique identifier for the budget entry
user_id (FK)	INT/UUID	References User
category_id (FK)	INT	References Category
month	DATE	Month the budget applies to
limit_amount	DECIMAL	Budget cap set by the student

Insight

Attribute	Type	Description
insight_id (PK)	INT	Unique identifier for the generated insight
user_id (FK)	INT/UUID	References User
month	DATE	Month the insight covers
summary_text	TEXT	AI-generated narrative summary
tip_text	TEXT	Actionable saving tip
generated_at	DATETIME	Timestamp of generation

Similarly, you can define other entities, relationships between entities, and methods representing activities on those entities.

Note: These are just examples; you do not have to adhere to these structures and can design your own table structure with different columns.

1.9 Project Deliverables

You will be required to design, build, and submit the project along with a complete project report that includes:

- Problem Definition
- Design Specifications
- Diagrams such as Flowcharts for various activities, Data Flow Diagrams, and so on
- Database Design
- Test Data Used in the Project
- **Project Installation Instructions (MANDATORY)**
- **User Credentials for all Types of Users with Passwords (MANDATORY)**

Documentation is considered a very important part of the project. Ensure that documentation is complete and comprehensive.

Documentation should not contain any source code.

The consolidated project will be submitted as a zip file with a ReadMe.doc file listing assumptions (if any) made at your end and SQL script files (.sql) OR schema files containing database and table definitions.

Note: Preferably, host the working Web application on a Website and share the URL for evaluation.

Submit a video (.mp4 file) demonstrating the working of the Web application, including all features under Functional Requirements. This is MANDATORY.

Over and above the given specifications, you can apply your creativity and logic to improve the system. Sitemap: To understand the flow of the **Campus Coin** Web application, you will have to create and add a sitemap to the home page of your application.

~~~ End of Document ~~~